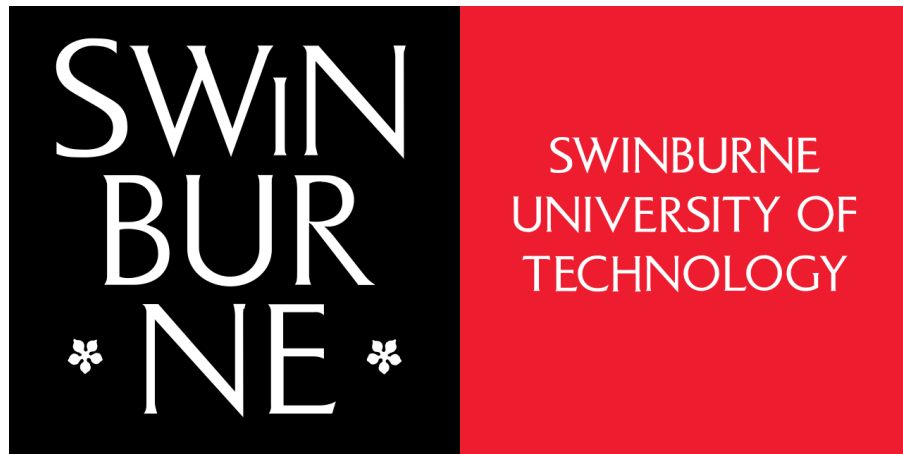




COS30018 – Intelligent Systems

Automated Fall Detection using AI Vision

Team Details

Sherlyn Jia Wen KOK – 102788463

Daniel Chuan Feng SIE – 102788159

Zwe Htet Zaw – 102783659

Jessica Jenny ANAK PAUL NEWMAN – 104404363

Table of Contents

1.	Introduction	3
2.	Dataset Collection and Annotation	3
2.1.	Dataset Summary	4
3.	System Architecture	4
3.1.	Repository Structure.....	4
3.2.	One-Step Fall Detection Architecture	4
3.3.	Two-Step Fall Detection Architecture	5
3.4.	Pose Extension Architecture (Bonus)	5
3.5.	GUI Module	5
4.	Implemented Machine Learning Techniques.....	6
4.1.	One-Step Model using YOLOv8	6
4.2.	Results.....	6
4.3.	Two-Step Model	6
4.4.	Results.....	7
5.	Extension Task 1 – Robust Low-Light Fall Detection	7
5.1.	Enhancement Modes.....	7
5.2.	Results.....	7
6.	Bonus Pose Estimation Extension.....	8
6.1.	Model Implementation	8
7.	Example of System Functionality.....	9
8.	Analysis of Implementation	9
8.1.	One-Step vs. Two-Step.....	9
8.2.	Condition Analysis and Limitations	10
8.3.	Practical Application Description.....	10
9.	Conclusion	11
10.	Presentation and Repository Link	12
10.1.	Presentation	12
10.2.	Repository	12
11.	Appendix.....	13
11.1.	Low Light Fall Detection Detailed Results	13
11.2.	Feature Extraction Table.....	13
11.3.	Classification Rules.....	13
11.4.	System Functionality Results Table	14
11.5.	System Functionality Summary Table	15

1. Introduction

Fall detection is one of the biggest health problems amongst the increasing number elderly people that live alone as falls lead to severe injuries and potentially cause death. Previously, traditional approaches relied on a wearable sensor-based device that, although having high detection rate, often has limitations due to user issues such as reluctance, discomfort or simply not remembering to put on the device. Therefore, this project proposes an automated fall detection system with AI vision that includes object detection and classification based on deep learning approaches.

This system will aim to automatically detect human falls from images or real-time video. It is designed around two architectures namely, one-step and two-step fall detection model. The one-step fall detection model will be trained with a single object detection that classifies humans' actions from detecting bounding boxes. The two-step detection model has two steps which are people detection followed by fall recognition. The objective is to evaluate both approaches and compare their performance under different circumstances such as environment, viewpoint, distance and more. A third low light detection and fourth pose estimation extension are also included as an extension task and bonus research task.

This project delivers the following components:

1. One-step fall detection baseline
2. Two-step fall detection pipeline
3. Extension 1 – Low-light Robustness
4. Pose-estimation extension
5. Live GUI
6. Evaluation Tooling

2. Dataset Collection and Annotation

This project uses the lecturer-provided fall detection dataset together with additional self-collected data. The provided training and validation images were merged into the training split, following the assignment requirement. Extra training images were also added to improve robustness across different poses, backgrounds, and camera conditions.

All annotations are stored in YOLO format, where each label contains the class ID and the normalised bounding box coordinates. The three classes used in this project are fall detected, walk, and sit, represented by class IDs 0, 1, and 2 respectively.

A separate held-out test set was self-collected by the team and was not used during training. This avoids overlap between training and testing images. Test images were captured using mobile phones and cameras in different indoor and outdoor locations, with variations in lighting, viewpoint, distance, and environment.

The final evaluated test split contains 502 images and 648 labelled person instances. The number of labelled instances is higher than the number of images because some images

contain multiple people. Each visible person was annotated with a bounding box and one of the three action labels.

To support condition-based analysis, each test image was also assigned metadata for lighting, viewpoint, distance, and environment. This allows the evaluation to compare performance under realistic scenarios, not only by overall accuracy.

2.1. Dataset Summary

Dataset Split	Images	Labelled Person Instances	Classes	Purpose
Training set	11,476	From YOLO labels	Fall / Walk / Sit	Model training
Test set	502	648	Fall / Walk / Sit	Final evaluation
Condition metadata	502	648	Fall / Walk / Sit	Condition analysis

Metadata Type	Categories
Lighting	Bright, Dim, Low Light, Normal, Shadowy
Viewpoint	Front, Back, Side, Angled, Top, Mixed
Distance	Close, Medium, Far, Very Far, Mixed
Environment	Indoor and outdoor locations such as rooms, stairs, walkways, library, park, and open spaces

3. System Architecture

The Automated Fall Detection System is designed as a modular vision pipeline that supports offline evaluation and real-time GUI demonstration. The system includes shared preprocessing, three inference pipelines, evaluation scripts, and a GUI frontend. All components are implemented in Python, with training outputs, evaluation results, prediction files, and report assets saved in the runs directory.

The overall pipeline begins with an input image, video frame, webcam stream, or image folder. The input can optionally pass through the low-light enhancement module before being sent to one of the selected inference pipelines: one-step YOLO, two-step detector-classifier, or pose-estimation extension. The output predictions are then used by the evaluation scripts for metric calculation or displayed in the GUI with bounding boxes, class labels, confidence scores, and fall-alert status.

3.1. Repository Structure

The codebase is organised into notebook pipelines and supporting Python scripts. The main project files are listed below.

File	Purpose
01_one_step_baseline_pipeline.ipynb	Trains and evaluates the one-step YOLO fall detection model
02_two_step_pipeline.ipynb	Builds the two-step detector and ResNet-18 crop classifier pipeline
03_model_comparison.ipynb	Compares one-step, two-step, and extension results
scripts/live_fall_gui.py	Implements the real-time GUI and fall alert system
scripts/low_light_enhancement.py	Contains the low-light enhancement methods
scripts/pose_fall_extension.py	Implements the pose-estimation rule-based fall detection extension
scripts/evaluate_conditions.py	Evaluates model performance by lighting, viewpoint, distance, and environment
scripts/evaluate_low_light_extension.py	Evaluates the low-light enhancement extension
scripts/evaluate_pose_extension.py	Evaluates the pose-estimation extension
scripts/tune_model_thresholds.py	Supports confidence threshold and crop-padding tuning

This structure separates training, evaluation, extension experiments, and real-time demonstration, making the system easier to maintain and test.

3.2. One-Step Fall Detection Architecture

The one-step pipeline uses a single YOLOv8 model trained on the combined fall detection dataset. The model performs both localisation and classification in one forward pass. For each image or video frame, it predicts bounding boxes and assigns each detected person to one of three classes: fall detected, walk, or sit.

The model is initialised from COCO-pretrained YOLOv8 weights and fine-tuned using transfer learning. COCO pretraining is useful because the model already has general

visual features for detecting objects and people before being adapted to the fall detection task. During inference, the model directly outputs the bounding box coordinates, class label, and confidence score for each detected instance.

This architecture is the simplest from a deployment perspective because it only requires one trained model file and one inference call. It is also well suited for the GUI because the detection and action classification happen together, reducing pipeline complexity and making real-time prediction easier to manage.

3.3. Two-Step Fall Detection Architecture

The two-step pipeline separates the task into person detection followed by action classification. Stage 1 uses a YOLOv8 person detector pretrained on COCO to locate people in the image. This stage focuses only on detecting human regions, not identifying the action class.

After a person is detected, the bounding box is cropped from the image. A small padding margin is added around the crop to preserve useful posture context, such as body orientation, surrounding floor area, and limb position. The crop is then resized and passed into Stage 2, a fine-tuned ResNet-18 classifier. The classifier predicts whether the detected person is fall detected, walk, or sit.

This architecture is more modular than the one-step model because the detection and classification stages can be improved separately. For example, the person detector could be replaced with a stronger detector, or the crop classifier could be retrained with more posture examples. However, the two-step design also introduces error propagation. If Stage 1 misses a person, creates an inaccurate bounding box, or removes too much context during cropping, Stage 2 cannot fully recover from that error. This makes the two-step pipeline useful for comparison, but also more sensitive to detection and cropping quality.

3.4. Pose Extension Architecture

The pose-estimation extension uses YOLOv8n-Pose to detect people and estimate 17 COCO human keypoints, including shoulders, hips, knees, ankles, and other body joints. Unlike the one-step and two-step models, this extension does not train a new classifier on the project dataset. Instead, it applies a rule-based fall recognition method using the geometry of the detected skeleton.

The rule system extracts features such as torso angle, bounding box width-to-height ratio, vertical keypoint spread, hip-to-knee distance, head-to-hip distance, number of visible keypoints, and average keypoint confidence. These features are used to classify posture

into fall detected, sit, or walk. For example, a fall is more likely when the body is more horizontal and spread close to the ground, while sitting tends to produce a more compact posture and walking is usually more upright.

This approach is useful as a bonus extension because it is explainable and does not require additional training data. It is less dependent on background texture than appearance-based classification because the decision is based on body structure. However, its performance depends on the visibility and accuracy of keypoints, so dark images, blur, occlusion, or unusual camera angles can still reduce reliability. The pose extension can also be combined with the low-light enhancement module to test whether clearer images improve keypoint detection.

3.5. GUI Module

The GUI is implemented in `scripts/live_fall_gui.py` using OpenCV. It connects the trained models and extension pipelines into a working real-time prototype. The GUI supports webcam input, video files, single images, and image folders. It also supports three selectable inference modes at launch: one-step, two-step, and pose.

The GUI architecture includes several components:

Component	Purpose
FrameSource	Handles webcam, video, image, or folder input
Enhancement pipeline	Applies optional low-light enhancement before inference
Inference dispatch	Sends each frame to the selected model pipeline
AlertState	Controls fall alert activation using consecutive fall frames
Dashboard composition	Draws the video display, status panel, labels, and metrics
Event logging	Saves optional frame-by-frame prediction logs

After launch, the GUI displays the annotated video feed on the left and a real-time status panel on the right. The video feed shows predicted bounding boxes or pose annotations with class labels and confidence scores. The status panel shows the current alert state, confirmation progress, FPS, object count, enhancement mode, luminance value, per-class detection counts, and recent predictions.

The fall alert is controlled by an AlertState mechanism. Instead of activating from a single fall prediction, the system can require fall detection across multiple consecutive frames. This reduces flickering and helps avoid unstable one-frame false alerts. Once a fall is

confirmed, the alert remains visible for a short hold duration. This GUI demonstrates that the project is not only an offline model evaluation, but also a functional prototype for real-time fall monitoring.

4. Implemented Machine Learning Techniques

This project implements two main deep learning architectures for fall detection: a one-step YOLOv8 object detection model and a two-step detector-classifier pipeline. Both models are evaluated on the same held-out self-collected test set to ensure a fair comparison. The evaluation focuses on fall detected, walk, and sit classes.

4.1. One-Step Model using YOLOv8

The one-step model uses YOLOv8n as a single object detection model. It is trained to directly detect people and classify their action in one forward pass. For each image or video frame, the model predicts a bounding box, class label, and confidence score for each detected person.

The model is initialised from COCO-pretrained YOLOv8 weights and fine-tuned on the prepared fall detection dataset. Transfer learning is used because the pretrained model already contains general visual features for detecting objects and people. Fine-tuning adapts these features to the project's three fall detection classes: fall detected, walk, and sit.

The model was trained for up to 100 epochs with early stopping. The Ultralytics training pipeline also applies built-in data augmentation, such as mosaic augmentation, image flipping, and colour variation. These augmentations help improve generalisation, especially because the test set contains different environments, viewpoints, distances, and lighting conditions.

The one-step architecture is efficient because detection and classification are handled by the same model. This makes it suitable for real-time GUI deployment, since only one model needs to be loaded and executed.

4.2. One-Step Results

The one-step model was evaluated using the Ultralytics validation pipeline on the held-out test set. The main metrics used were precision, recall, mAP@50, and mAP@50-95.

Class	Precision	Recall	mAP@50	mAP@50-95
fall detected	0.766	0.744	0.789	0.522
walk	0.848	0.909	0.907	0.662
sit	0.681	0.706	0.694	0.439
Mean	0.765	0.786	0.797	0.541

The results show that walk is the strongest class, with the highest recall and mAP@50. This is expected because walking and standing postures are visually more distinct from the other classes. Fall detected performs reasonably well, but its recall is lower than walk, showing that some fall cases are still missed. Sit is the weakest class overall, mainly because sitting and falling can appear visually similar when a person is close to the floor or captured from certain viewpoints.

4.3. Two-Step Model

The two-step model separates fall detection into two stages. Stage 1 uses a YOLOv8n person detector pretrained on COCO to locate people in the image. Stage 2 uses a ResNet-18 classifier to classify each detected person crop as fall detected, walk, or sit.

After Stage 1 detects a person, the bounding box is cropped from the image with a small padding margin. This padding helps preserve body posture context around the person. The crop is then resized and passed into the ResNet-18 classifier. The classifier outputs class probabilities, and the class with the highest probability becomes the final prediction.

This architecture is useful because it separates localisation and action recognition. The detector focuses only on finding people, while the classifier focuses on recognising posture from cropped person images. However, the two-step pipeline is also more sensitive to error propagation. If the person detector misses a person or creates a poor crop, the classifier cannot correct that mistake.

From 11,476 training images, 16,008 training crops were generated for classifier training. From the 502 test images, 648 test crops were generated because some images contain multiple people.

4.4. Results

The two-step pipeline was evaluated end-to-end using the same held-out test set. The crop classifier achieved 80.6% accuracy on the held-out test crops, but the full pipeline performance depends on both person detection and crop classification.

Class	Prediction	Recall	F1	TP	FP	FN
fall detected	0.770	0.595	0.671	94	28	64
walk	0.842	0.869	0.855	218	41	33
sit	0.649	0.791	0.713	189	102	50
Mean	0.746	0.773	0.759	501	171	147

The two-step model achieved an overall F1 score of 0.759. Walk again performs best, while fall detected has the lowest recall. This means the two-step model misses more fall cases compared with the other classes. A likely reason is that fall postures can be harder for the

person detector to detect correctly because the body may be horizontal, partially occluded, or close to the ground.

The sit class has higher recall but lower precision, meaning the model often predicts sit, including some cases that should be fall. This shows the main difficulty of the task: fall and sit can have similar visual appearances, especially when a person is sitting on the floor or lying in a curled posture.

Overall, the one-step YOLO model slightly outperforms the two-step pipeline. This suggests that, for this dataset, learning bounding box localisation and action classification together is more effective than separating the task into detection and crop classification. However, the two-step model remains useful as a comparison architecture because it is modular and allows the person detector and posture classifier to be improved separately.

5. Extension Task 1 – Robust Low-Light Fall Detection

A practical fall detection system must continue working when lighting conditions are poor, especially in home or care environments where falls may occur at night or in dim rooms. Low-light images can reduce model performance because important visual features such as body outline, limb position, floor contact, and object boundaries become less clear. This can increase missed falls or confusion between similar classes such as fall and sit.

To address this, the project implements a low-light robustness extension using an inference-time image enhancement pipeline. Instead of retraining the detection models, the input frame is enhanced before being passed into the one-step or two-step model. This allows the same trained models to be tested under different preprocessing strategies and makes the extension compatible with both architectures.

The enhancement module is implemented in `scripts/low_light_enhancement.py`. It supports five modes: none, gamma correction, CLAHE, gamma-CLAHE, and auto. These modes were compared to determine whether improving image brightness and contrast could improve fall detection performance under challenging lighting conditions.

5.1. Enhancement Modes

Enhancement Mode	Description
None	The original image is passed directly into the model without preprocessing. This is used as the baseline.
Gamma correction	Brightens darker regions by adjusting the image intensity curve. This helps reveal details in underexposed frames.

CLAHE	Contrast Limited Adaptive Histogram Equalisation improves local contrast without over-brightening the whole image.
Gamma-CLAHE	Applies gamma correction first, followed by CLAHE. This combines brightness correction with local contrast enhancement.
Auto	Computes frame luminance and applies gamma-CLAHE only when the image is below the low-light threshold. Well-lit images are left unchanged.

The auto mode uses the mean luminance of the image to decide whether enhancement is needed. This is useful for real-world deployment because lighting may change during live monitoring. Applying enhancement only when necessary reduces the risk of degrading frames that are already clear.

5.2. Results And Analysis

The low-light extension was evaluated by comparing model performance across the different enhancement modes. F1 score was used as the main comparison metric because it balances precision and recall, which is important for fall detection where both false alarms and missed falls are undesirable.

Enhancement Mode	One-Step F1	Two-Step F1	Notes
None	0.704	0.716	No preprocessing baseline
Gamma	0.776	0.774	Improves dark image visibility
CLAHE	0.780	0.775	Improves local contrast
Gamma-CLAHE	0.808	0.797	Strong combined enhancement
Auto	0.810	0.797	Best deployment option for mixed lighting

The results show that image enhancement improves performance for both architectures. For the one-step model, F1 score increased from 0.704 with no enhancement to 0.810 with auto enhancement. For the two-step model, F1 score increased from 0.716 to 0.797. This suggests that low-light preprocessing helps the models recover visual details that are important for detecting human posture.

Gamma correction and CLAHE both improved performance individually, but the combined gamma-CLAHE and auto modes achieved the strongest results. This is because gamma correction improves image brightness, while CLAHE improves local contrast around body boundaries and limbs. Together, they make the person's posture more visible to the detection models.

Auto mode is the most suitable option for deployment because it only applies enhancement when the frame is dark. This prevents unnecessary processing on normal-light frames and reduces the chance of over-enhancing already clear images. Therefore, the low-light extension improves robustness while remaining practical for the real-time GUI system.

However, enhancement does not fully solve all low-light problems. If the image is extremely dark, blurry, or occluded, the model may still miss the fall or confuse it with sitting. This means image enhancement is useful as a robustness layer, but future work could further improve performance by adding more low-light training data or training a model directly on enhanced and low-light examples. Detailed results can be found [here](#).

6. Bonus Pose Estimation Extension

Beyond the required Extension 1, the project implements a third fall detection pipeline using skeleton pose estimation as a bonus contribution. This pipeline uses YOLOv8n-Pose to extract 17 COCO keypoints from each detected person, then applies a transparent geometric rule engine to classify the posture which requires no additional training data.

The appearance-based models (YOLOv8 and ResNet-18) learn to classify fall postures from pixel patterns, which makes them sensitive to camera viewpoint, clothing, and image quality. A pose-based approach instead analyses the geometric arrangement of body joints which is less likely to be affected by these factors.

6.1. Model Implementation

This pose estimation uses YOLOv8n-Pose, also pretrained on COCO like the rest of the models in this project. No fine tuning is done and the model will automatically detect persons in frame using COCO skeleton layout. The model also utilizes feature extraction to compute geometric features from each detected person's keypoints and bounding box which can be found in the [table](#). Moreover, the pose extension pipeline is fully compatible with all 5 low light enhancement modes, evaluation is also made and results are exported to the runs folder. This model also has its advantages such as no training is required, it is fully explainable, is robust in terms of viewpoint and supported by GUI. Its limitations are that it degrades in darker frames and it is not directly comparable to YOLO mAP as it is evaluated using the same pipeline as two-step detection. There are also classification rules applied such as fallback, fall detected, sit and walk which can be referred [here](#).

Keypoint Index	Body Part	Keypoint Index	Body Part
----------------	-----------	----------------	-----------

0	Nose	9	Left Wrist
1-4	Eyes and Ears	10	Right Wrist
5	Left Shoulder	11	Left Hip
6	Right Shoulder	12	Right Hip
7	Left elbow	13	Left knee
8	Right Elbow	14-16	Knees & Ankles

7. Example of System Functionality

This section details the system's behavior across scenarios tested during evaluation. First, the GUI can be launched from the command line with different modes.

1. `python scripts/run_gui.py --mode one-step --source 0`
2. `python scripts/run_gui.py --mode two-step --source 0`
3. `python scripts/run_gui.py --mode pose --source 0`
4. `python scripts/run_gui.py --mode two-step --source 0 --enhancement auto` (OR 'E' key when toggled live)

After launching the GUI, the annotated video feed is displayed on the left with color coded (red, green and orange) bounding boxes. The status panel on the right shows the current alert state, FPS, object counts per class, current enhancement mode, and more. The automated fall detection system can be applied to various scenarios such as a fall in an indoor environment with normal lighting, multiple persons detection simultaneously, low light environments in varying angles and distance.

In a normal indoor environment, both the one-step and two-step pipelines successfully detected a fall after a person transition from walking to falling using AlertState to activate fall alert. As for multiple person detection, both one and two-step processes the detected object and will prioritize a fall event when more than one class happens simultaneously. Under low light conditions, usually the system will apply gamma-CLAHE enhancement mode to improve frame visibility and improve detection efficiency. The system can also handle different viewpoints and distances as it was tested across these scenarios. It performs normally close to medium ranges, but confidence will reduce once the object becomes further away from the camera or when vision is partially obstructed by other items in the frame. Overall, these scenarios show that the system is robust and can handle fall detection reliably across different conditions.

Further condition analysis was also implemented which generates conditions across lighting, viewpoint, distance, and environment can be seen in the [results](#) and [summary](#) table attached.

8. Analysis of Implementation

8.1. One-Step vs. Two-Step

Metric	One-Step YOLO	Two-Step
--------	---------------	----------

Precision	0.765	0.746
Recall	0.786	0.773
F1	0.775	0.759
mAP@50	0.797	Non-Applicable
Crop-level accuracy	Non-Applicable	80.6% on 648 crops
Inference stages	1	2
Model files required	1	2 (detector & classifier)

The one-step YOLO model achieved a slightly higher overall F1 score than the two-step pipeline. This shows that, for this dataset, learning object localisation and action classification together was more effective than separating the task into two stages.

The two-step model still performed reasonably well, especially at the crop-classification level, where the ResNet-18 classifier achieved 80.6% accuracy. However, the final end-to-end result is lower because the pipeline depends on both stages working correctly. If the person detector misses a person, creates an inaccurate bounding box, or removes useful body context during cropping, the classifier cannot recover from that error. This is known as error propagation and is a key limitation of the two-step architecture.

Walk was the easiest class for both models because walking usually has a clear upright body posture. Fall and sit were more difficult to separate because some sitting postures, especially sitting on the floor, can visually resemble a fall. This explains why fall recall is lower in the two-step model and why sit has more false positives.

Overall, the one-step model is more suitable for real-time deployment because it is simpler, faster, and only requires one model. The two-step model remains useful as a comparison because it is modular and allows the detector and classifier to be improved separately.

8.2. Condition Analysis and Limitations

The models were also evaluated under different lighting, viewpoint, distance, and environment conditions. This is important because a fall detection system must work reliably outside controlled training conditions.

For lighting, performance was generally better in normal or well-lit conditions and weaker in low-light conditions. The one-step model achieved F1 0.793 under normal lighting but dropped to 0.739 in low light. The two-step model achieved F1 0.791 under normal lighting but dropped further to 0.684 in low light. This shows that the two-step model is more sensitive to poor lighting, likely because low-light affects both person detection and crop classification.

For viewpoint, mixed and angled viewpoints performed well, while front and top viewpoints were more challenging. Top-view and unusual angles can change the apparent body shape, making it harder to distinguish falling from sitting or lying down.

This suggests that more training data from overhead and unusual camera angles would improve robustness.

For distance, medium and far examples performed reasonably well, while close and very far examples were more difficult. Very far subjects contain fewer visual details, while close subjects may be partially cropped or have less surrounding context. Both cases reduce the model's ability to understand full body posture.

For environment, indoor room and outdoor park examples performed reasonably well because they had more samples and clearer human visibility. Smaller categories such as stairs, library, and some outdoor locations were less reliable because they had fewer examples or more complex backgrounds. Stairs are especially challenging because body posture, object boundaries, and fall/sit positions can overlap visually.

The main limitation of the project is the difficulty of separating fall and sit classes. This is not only a model issue but also a data issue, because some postures are visually ambiguous in a single image. Another limitation is that the system is frame-based, so it does not yet use motion over time. Future work could improve this by using video sequences or temporal models to detect the transition from standing or walking into a fall.

8.3. Practical Application Description

In a real home or care facility, the system could be used as a camera-based assistive monitoring tool for elderly people or patients at risk of falling. The GUI demonstrates the basic practical workflow by showing live predictions and displaying an alert when a fall is detected.

However, real deployment would require stronger reliability and safety considerations. False positives could cause unnecessary alerts, while false negatives could miss real emergencies. Therefore, the alert mechanism should be connected to audio alarms, caregiver notifications, or mobile alerts rather than only a visual banner.

Privacy is also important because the system uses live video. Any real deployment should require consent, secure storage, and careful handling of recorded images or videos. Camera placement would also affect performance, so cameras should be positioned to reduce occlusion and cover likely fall areas.

The low-light extension improves practicality because falls may occur at night or in dim rooms. The auto enhancement mode is useful because it only applies enhancement when needed, making it more suitable for mixed lighting conditions. Overall, the prototype shows that AI vision can support fall monitoring, but further work is needed before it can be used as a safety-critical real-world system.

9. Conclusion

This project successfully implemented an automated fall detection system using AI vision. The system includes a one-step YOLOv8 fall detection model, a two-step detector-classifier pipeline, condition-based evaluation, a self-collected test set, a real-time GUI with fall alert functionality, and a low-light robustness extension. A bonus pose-estimation extension was also implemented to provide an explainable posture-based comparison.

The one-step YOLO model achieved the best overall result, with an F1 score of 0.775 and mAP@50 of 0.797. The two-step pipeline achieved an F1 score of 0.759 and a crop-level classifier accuracy of 80.6%. Although the two-step approach is modular and useful for comparison, its final performance was slightly lower because errors from the person detection stage can affect the classifier stage.

The low-light extension showed the strongest research contribution. By applying image enhancement before inference, performance improved under challenging lighting conditions. Auto enhancement and gamma-CLAHE produced the best results, showing that brightness and contrast correction can help the models detect human posture more reliably in darker scenes.

The pose-estimation extension added an explainable alternative by using YOLOv8-Pose keypoints and rule-based body geometry analysis. Although it was not the best-performing method overall, it provided useful insight into how body structure can support fall recognition.

Overall, the project satisfies the main requirements by implementing and comparing one-step and two-step fall detection, analysing performance under different conditions, and demonstrating a working GUI prototype with live prediction and fall alert. Future work could improve the system by collecting more diverse fall examples, adding more overhead and low-light training data, using temporal video features for early fall detection, and integrating audio or mobile alerts for practical deployment.

10. Presentation and GitHub Link

10.1. Presentation Link

10.2. GitHub Link

<https://github.com/shr0914/intelligent-system>

11. Appendix

11.1. Low Light Fall Detection Detailed Results

	model	enhancement	class_id	class_name	tp	fp	fn	precision	recall	f1
1	one-step YOLO	auto	,all	,248,65,51	0.792332268370607	0.8294314381270903	0.8104575163398692			
2	one-step YOLO	clahe	,all	,241,78,58	0.7554858934169278	0.8060200668896321	0.7799352750809062			
3	one-step YOLO	gamma	,all	,237,75,62	0.7596153846153846	0.7926421404682275	0.7757774140752863			
4	one-step YOLO	gamma-clahe	,all	,248,67,51	0.7873015873015873	0.8294314381270903	0.8078175895765471			
5	one-step YOLO	none	,all	,216,99,83	0.6857142857142857	0.7224080267558528	0.7035830618892508			
6	two-step detector + classifier	auto	,all	,248,75,51	0.7678018575851393	0.8294314381270903	0.797427652733119			
7	two-step detector + classifier	clahe	,all	,243,85,56	0.7408536585365854	0.8127090301003345	0.7751196172248803			
8	two-step detector + classifier	gamma	,all	,241,83,58	0.7438271604938271	0.8060200668896321	0.7736757624398073			
9	two-step detector + classifier	gamma-clahe	,all	,248,75,51	0.7678018575851393	0.8294314381270903	0.797427652733119			
10	two-step detector + classifier	none	,all	,222,99,77	0.6915887850467289	0.7424749163879598	0.7161290322580646			
11										

11.2. Feature Extraction Table

Feature	Description	Used For
torse_angle_from_horizontal	Angle between shoulder midpoint and hip midpoint, measured from horizontal	Fall Detection
box_ratio (width/height)	Bounding box width divided by height	Fall and Sit Detection
vertical_spread	Normalised vertical extent of all visible keypoints relative to box height	Fall Detection
hip_knee_gap	Normalised vertical distance between hip midpoint and knee midpoint	Sit Detection
head_hip_gap	Normalised vertical distance between head and hip midpoint	Sit Detection
visible_keypoints	Count of keypoints above min_keypoint_conf threshold (0.25)	Rule gating
avg_keypoint_conf	Mean confidence of visible keypoints	Confidence scaling

11.3. Classification Rules

Fallback (< 5 visible keypoints)

```
def classify_pose(features: dict[str, float], config: PoseRuleConfig) -> tuple[int, float, str]:
    if features["visible_keypoints"] < config.min_visible_keypoints:
        class_id = 0
        if features["box_ratio"] >= config.fall_ratio_threshold else 1
        confidence = min(0.65, 0.35 + abs(features["box_ratio"] - 1.0) * 0.20)
        return class_id, confidence, "fallback_box_shape"
```

Fall Detected

```
horizontal_torso = features["torso_angle_from_horizontal"] <= config.fall_angle_threshold
wide_body = features["box_ratio"] >= config.fall_ratio_threshold
compact_vertical_pose = features["vertical_spread"] <= config.fall_vertical_spread_threshold
fall_score = sum([horizontal_torso, wide_body, compact_vertical_pose])
if fall_score >= 2:
    confidence = min(0.98, 0.52 + 0.12 * fall_score + 0.18 * features["avg_keypoint_conf"])
    return 0, confidence, "horizontal_or_wide_pose"
```

Sit

```
likely_sit = features["box_ratio"] >= config.sit_ratio_threshold and (  
    features["hip_knee_gap"] <= config.sit_knee_hip_threshold  
    or features["head_hip_gap"] <= 0.40  
)  
if likely_sit:  
    confidence = min(0.94, 0.48 + 0.28 * features["avg_keypoint_conf"])  
    return 2, confidence, "compact_seated_pose"
```

Walk

```
confidence = min(0.94, 0.50 + 0.30 * features["avg_keypoint_conf"])  
return 1, confidence, "upright_pose"
```

11.4. System Functionality Results Table

Model	Condition	Precision	Recall	F1
One-Step YOLO	Bright	0.667	0.571	0.615
One-Step YOLO	Dim	0.810	0.810	0.810
One-Step YOLO	Low light	0.739	0.739	0.739
One-Step YOLO	Normal	0.821	0.766	0.793
One-Step YOLO	Shadowy	0.722	0.803	0.760
Two-Step ResNet-18	Bright	0.833	0.714	0.769
Two-Step ResNet-18	Dim	0.772	0.810	0.790
Two-Step ResNet-18	Low light	0.667	0.703	0.684
Two-Step ResNet-18	Normal	0.780	0.804	0.791
Two-Step ResNet-18	Shadowy	0.740	0.761	0.75

1	model, lighting, class_id, class_name, tp, fp, fn, precision, recall, f1		
2	one-step YOLO, bright, 1, walk, 1, 0, 0, 1.0, 1.0, 1.0		
3	one-step YOLO, bright, 2, sit, 3, 2, 3, 0.6, 0.5, 0.5454545454545454		
4	one-step YOLO, dim, 0, fall detected, 10, 3, 3, 0.7692307692307693, 0.7692307692307693, 0.7692307692307693		
5	one-step YOLO, dim, 1, walk, 24, 2, 3, 0.9230769230769231, 0.8888888888888888, 0.9056603773584906		
6	one-step YOLO, dim, 2, sit, 17, 7, 6, 0.7083333333333334, 0.7391304347826086, 0.723404255319149		
7	one-step YOLO, low_light, 0, fall detected, 20, 2, 24, 0.9090909090909091, 0.4545454545454545, 0.6060606060606061		
8	one-step YOLO, low_light, 1, walk, 63, 4, 13, 0.9402985074626866, 0.8289473684210527, 0.8811188811188811		
9	one-step YOLO, low_light, 2, sit, 39, 37, 6, 0.5131578947368421, 0.8666666666666667, 0.6446280991735538		
10	one-step YOLO, normal, 0, fall detected, 59, 19, 16, 0.7564102564102564, 0.7866666666666666, 0.7712418300653594		
11	one-step YOLO, normal, 1, walk, 115, 16, 11, 0.8778625954198473, 0.9126984126984127, 0.8949416342412452		
12	one-step YOLO, normal, 2, sit, 88, 22, 53, 0.8, 0.624113475177305, 0.701195219123506		
13	one-step YOLO, shadowy, 0, fall detected, 22, 8, 4, 0.7333333333333333, 0.8461538461538461, 0.7857142857142856		
14	one-step YOLO, shadowy, 1, walk, 21, 13, 0, 0.6176470588235294, 1.0, 0.7636363636363637		
15	one-step YOLO, shadowy, 2, sit, 14, 1, 10, 0.9333333333333333, 0.5833333333333334, 0.7179487179487181		
16	two-step detector + classifier, bright, 1, walk, 1, 0, 0, 1.0, 1.0, 1.0		
17	two-step detector + classifier, bright, 2, sit, 4, 1, 2, 0.8, 0.6666666666666666, 0.7272727272727272		
18	two-step detector + classifier, dim, 0, fall detected, 7, 2, 6, 0.7777777777777778, 0.5384615384615384, 0.6363636363636364		
19	two-step detector + classifier, dim, 1, walk, 24, 2, 3, 0.9230769230769231, 0.8888888888888888, 0.9056603773584906		
20	two-step detector + classifier, dim, 2, sit, 20, 11, 3, 0.6451612903225806, 0.8695652173913043, 0.7407407407407407		
21	two-step detector + classifier, low_light, 0, fall detected, 16, 3, 28, 0.8421052631578947, 0.3636363636363636, 0.5079365079365079		
22	two-step detector + classifier, low_light, 1, walk, 59, 4, 17, 0.9365079365079365, 0.7763157894736842, 0.8489208633093525		
23	two-step detector + classifier, low_light, 2, sit, 41, 51, 4, 0.44565217391304346, 0.9111111111111111, 0.5985401459854015		
24	two-step detector + classifier, normal, 0, fall detected, 50, 16, 25, 0.7575757575757576, 0.6666666666666666, 0.7092198581560282		
25	two-step detector + classifier, normal, 1, walk, 116, 28, 10, 0.8055555555555556, 0.9206349206349206, 0.8592592592592592		
26	two-step detector + classifier, normal, 2, sit, 109, 34, 32, 0.7622377622377622, 0.7730496453900709, 0.7676056338028169		
27	two-step detector + classifier, shadowy, 0, fall detected, 21, 7, 5, 0.75, 0.8076923076923077, 0.7777777777777779		
28	two-step detector + classifier, shadowy, 1, walk, 18, 7, 3, 0.72, 0.8571428571428571, 0.782608695652174		
29	two-step detector + classifier, shadowy, 2, sit, 15, 5, 9, 0.75, 0.625, 0.6818181818181818		

11.5. System Functionality Summary Table

Class	One-Step Best	One-Step Worst	Two-Step Best	Two-Step Worst
Walk	Bright (1.0)	Low-light (0.88)	Bright (1.0)	Shadowy (0.78)
Sit	Shadowy (0.72)	Bright (0.55)	Normal (0.77)	Low-light (0.60)
Fall	Normal (0.77)	Low-light (0.61)	Shadowy (0.78)	Low-light (0.51)